

Investigative Checklist for DOCX Metadata and OOXML Signals of Plagiarism or AI-Generated Content

Executive summary

A `.docx` file is not a monolithic document. It is an Open Packaging Conventions package—a ZIP-based container of XML parts, media, relationships, and document properties. That structure is what makes DOCX useful for investigation: potentially relevant evidence is distributed across `docProps/*`, `word/*.xml`, `_rels/*.rels`, media folders, and package timestamps rather than being confined to visible body text alone. ¹

The strongest DOCX indicators are not usually “AI fingerprints” in any narrow sense. They are cross-part inconsistencies that suggest imported text, third-party editing, templated assembly, copied source material, or late-stage repackaging: mismatched author names, implausible creation/modification chronology, abnormally low revision/edit-time values for long documents, preserved tracked-change or comment authors, `altChunk` import anchors, RSID patterns showing whole paragraphs appearing in a single session, orphaned or foreign headers/footers/styles, and media or custom XML that point back to another source or workflow. ²

For plagiarism or contract cheating, the highest-value red flags are preserved third-party author/comment/revision data, explicit import artefacts such as `w:altChunk`, or shared RSID roots/patterns across suspiciously related documents. For AI assistance, the evidence is usually weaker and more indirect: automation residue, generic templating/custom XML, app/template strings inconsistent with the claimed workflow, unusually uniform single-session assembly, or machine-like accessibility metadata. Those clues should trigger further enquiries, not definitive findings. ³

The practical rule is cumulative assessment. One yellow flag is often just noise. Several independent yellow flags across different parts of the package justify escalation. A preserved third-party identity, unexplained imported-content relationship, or same-source RSID evidence is materially stronger and should be treated as a red flag requiring comparison with exemplar documents, submission-system data, and, where appropriate, a procedural interview. ⁴

Evidential position

This report frames DOCX artefacts as investigative prompts, not proof. That distinction matters. The National Institute of Standards and Technology ⁵ notes that digital forensic methods have limits, that technology changes can alter the meaning of artefacts, and that an examiner must understand uncertainty, peer review, tool limitations, and the possibility of deliberate obfuscation. In plain terms: a DOCX package can preserve telling clues, but those clues must be interpreted in context and corroborated. ⁶

A useful working model is this. A **yellow flag** is suspicious but readily explainable: it should prompt comparison against known-good documents from the same course, institution, template, and software

environment. A **red flag** is harder to explain innocently because it preserves evidence of prior authorship, explicit import, or shared origin. Confidence in this report means confidence that the artefact is a worthwhile trigger for further investigation, not confidence that misconduct is already proved. ⁷

One further caution is crucial for AI allegations. DOCX forensics is much better at showing **how a file was assembled or edited** than **who intellectually authored the prose**. Today's document-level metadata and OOXML structures can often support findings of copy/paste assembly, templating, external import, hidden text, or third-party revision history. They are far less reliable as standalone evidence that a language model generated the text. ⁸

Investigative workflow

A defensible workflow starts by preserving the submitted file, hashing it, and inspecting it read-only as a ZIP/OPC package. From there, work outward from low-cost, high-yield artefacts: package/core/extended/custom properties first, then settings and revision markers, then separate stories and peripheral parts, then media/relationships/custom XML, and only then compare anomalies against exemplars and the submission context. That order matches both the OPC structure and general forensic good practice. ⁹

```
flowchart TD
    A[Preserve original DOCX] --> B[Hash and work on a copy]
    B --> C[List ZIP package parts and member timestamps]
    C --> D[Read core, extended, and custom properties]
    D --> E[Inspect settings, comments, revisions, RSIDs]
    E --> F[Inspect document.xml plus styles, numbering, footnotes/endnotes, headers/footers]
    F --> G[Inspect media, alt text, embedded objects, relationships, customXml]
    G --> H[Correlate anomalies across independent parts]
    H --> I{Single weak clue or multiple consistent clues?}
    I -->|Single weak clue| J[Record as yellow flag and compare with known-good exemplar]
    I -->|Multiple consistent clues| K[Escalate as red flag for further inquiry]
    K --> L[Compare with other submissions, platform logs, and assessment context]
```

In practice, the workflow should produce a short evidential schedule: artefact, where found, why it matters, whether it is a yellow or red flag, how likely innocent explanations are, and what comparison or interview question it triggers. That makes the exercise proportionate and auditable instead of turning every stale metadata field into an allegation. ¹⁰

Flag catalogue

Package envelope and timestamps

Yellow flag — package/member timestamps are oddly compressed or uniform. Because DOCX maps to a ZIP package, each package member can carry a last-modified timestamp. ZIP stores those

dates in MS-DOS format with coarse precision, and a resave or repack can rewrite many entries at once. So “all parts stamped within a couple of seconds” is suspicious only as a packaging clue, not as proof of anything. It becomes more interesting when paired with low revision counts, extremely short edit time, or a late submission window. **Detect:** inspect ZIP central-directory times and compare the timestamps of `docProps/*`, `word/document.xml`, `word/styles.xml`, media, and relationship parts.

Confidence: low on its own. ¹¹

Red flag — document chronology conflicts with assessment reality. A creation date before the assignment release, or a long document with a creation/last-modified pattern suggesting near-instant assembly immediately before submission, is materially stronger. The higher-education forensic literature treats “author mismatch”, “creation date ahead of assignment release”, and “very low editing time or revision count in a long document” as worth escalation. **Detect:** correlate `dcterms:created`, `dcterms:modified`, `cp:revision`, `ap:TotalTime`, and the package timestamps against the assignment timeline. **Confidence:** medium to high when several fields agree. ¹²

Representative metadata pattern:

```
<!-- docProps/core.xml -->
<cp:coreProperties xmlns:cp="http://schemas.openxmlformats.org/package/2006/
metadata/core-properties"
                  xmlns:dc="http://purl.org/dc/elements/1.1/"
                  xmlns:dcterms="http://purl.org/dc/terms/"
                  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <dc:creator>someone_else</dc:creator>
  <cp:lastModifiedBy>candidate123</cp:lastModifiedBy>
  <cp:revision>1</cp:revision>
  <dcterms:created  xsi:type="dcterms:W3CDTF">2026-01-05T09:00:00Z</
dcterms:created>
  <dcterms:modified xsi:type="dcterms:W3CDTF">2026-05-07T23:58:00Z</
dcterms:modified>
</cp:coreProperties>
```

This is a synthetic example, but the fields and their meanings are standard package properties. ¹³

Core, extended, and custom properties

Yellow flag — weak or generic core identity fields. Core properties include creator, last modifier, title, subject, description, keywords, language, created, modified, revision, and version. Generic values such as blank creator plus a nonblank last modifier, or an environment-specific identifier that looks like a username or email unrelated to the submitter, are not conclusive but are worth comparison. Because the creator/lastModifiedBy identifiers are explicitly environment-specific, they can legitimately contain names, emails, or internal IDs. **Detect:** inspect `docProps/core.xml` directly or with a metadata extractor. **Confidence:** medium when inconsistent with the student’s normal naming pattern; otherwise low. ¹³

Yellow flag — extended-property ratios that do not fit the visible document. The extended-properties part can record application name/version, template name, total edit time, company, pages, words, paragraphs, and “shared document” state. A 3,000-word essay with `TotalTime` close to zero, minimal revision count, and a late final modification is a classic suspicion pattern. Still, these fields are

application-maintained rather than keystroke logs, so they are better treated as prompt-and-corroborate evidence. **Detect:** inspect `docProps/app.xml` and compare `Words`, `Pages`, `Paragraphs`, `TotalTime`, `Application`, `AppVersion`, `Template`, and `Company`. **Confidence:** medium in combination; low alone. ¹⁴

Yellow to red flag — application/template/workflow mismatch. `Application`, `AppVersion`, and `Template` are legitimate metadata elements. In a student context, a template or company string from an external organisation, workplace, publisher, or unrelated department can indicate borrowed origin. So can a highly specific template name not used by the institution. Likewise, an application string inconsistent with the claimed workflow can be relevant, although not inherently suspicious. **Detect:** compare `ap:Application`, `ap:AppVersion`, `ap:Template`, and `ap:Company` against the student's normal environment and any official departmental template. **Confidence:** medium if corroborated by headers/styles/customXml from the same external source. ¹⁵

Yellow flag — unexpected custom properties or document-management XML. Custom file properties are explicitly user-defined. Well-defined custom XML parts may also store document-management metadata, SharePoint content types, or other business data. In an academic submission, unexpected management metadata, content-type IDs, or external organisational property sets can suggest that the file started life elsewhere. On the other hand, many institutional templates legitimately include such parts. **Detect:** inspect `docProps/custom.xml` and `customXml/item*.xml`; identify part purpose by namespace, not just by filename. **Confidence:** medium when the namespace or property names clearly point to another organisation or workflow. ¹⁶

Representative extended/custom-property pattern:

```
<!-- docProps/app.xml -->
<Properties xmlns="http://schemas.openxmlformats.org/officeDocument/2006/
extended-properties"
            xmlns:vt="http://schemas.openxmlformats.org/officeDocument/2006/
docPropsVTypes">
  <Template>Normal.dotm</Template>
  <TotalTime>1</TotalTime>
  <Words>3021</Words>
  <Application>Example Generator</Application>
  <AppVersion>16.0000</AppVersion>
  <Company>External Org</Company>
</Properties>

<!-- docProps/custom.xml -->
<Properties xmlns="http://schemas.openxmlformats.org/officeDocument/2006/
custom-properties"
            xmlns:vt="http://schemas.openxmlformats.org/officeDocument/2006/
docPropsVTypes">
  <property fmtid="{D5CDD505-2E9C-101B-9397-08002B2CF9AE}" pid="2"
name="ContentTypeId">
    <vt:lpwstr>external-template-token</vt:lpwstr>
  </property>
</Properties>
```

The element names are standard; the values here are illustrative. 17

Revision history, comments, and author residue

Red flag — tracked revisions or comments preserve a third-party author. OOXML stores comment authors/initials and revision authors/dates directly in XML. If a file still contains comments or `w:ins` / `w:del` revisions naming another person, contractor, tutor, or collaborator, that is one of the strongest direct authorship anomalies available in DOCX. **Detect:** inspect `word/comments.xml` and any revision markup in `word/document.xml`, `footnotes.xml`, `endnotes.xml`, headers, and footers.

Confidence: high. 18

Yellow flag — absence of author/date data because the file has been scrubbed. The settings part can instruct the application to remove personal information and remove dates from annotations upon save. That means “no author names left” is not exonerating if `w:removePersonalInformation` and `w:removeDateAndTime` are enabled. **Detect:** inspect `word/settings.xml` for those flags before treating missing author/date metadata as meaningful. **Confidence:** high as a cautionary qualifier. 19

Red flag — RSID pattern suggests bulk insertion or shared origin. The OOXML settings part can hold `w:rsidRoot` and a list of RSIDs; RSID values identify editing sessions. The official documentation states that `rsidRoot` is associated with the first editing session and must remain identical across copies of the same original document. For plagiarism/contract-cheating triage, the strongest use is comparative: if two suspicious documents share the same `rsidRoot` or a distinctive RSID pattern, that supports common origin. Within one document, large uninterrupted text blocks with very few edit-session markers can suggest paste-in assembly rather than iterative composition. **Detect:** inspect `word/settings.xml` and RSID-bearing attributes in `document.xml`; compare against other documents where appropriate. **Confidence:** high for common-source comparison; medium for “bulk insertion” inference within a single file. 20

Representative revision-history pattern:

```
<!-- word/document.xml -->
<w:p xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main">
  <w:r><w:t>Existing text</w:t></w:r>
  <w:ins w:id="0" w:author="Third Party" w:date="2026-05-01T11:20:00Z">
    <w:r><w:t> inserted passage</w:t></w:r>
  </w:ins>
</w:p>

<!-- word/comments.xml -->
<w:comment xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main"
  w:id="1" w:author="Reviewer Name" w:initials="RN"/>
```

Again, these are representative element forms, not a copied specimen. 21

`document.xml`, hidden import clues, language, styles, and numbering

Red flag — explicit external-content import via `w:altChunk`. This is one of the cleanest structural indicators available. `altChunk` explicitly marks a location where external content is imported into the

document, and the corresponding relationship points to an import part. Word supports imported DOCX, templates, RTF, XML, XHTML, HTML, plain text, and even RFC822 message content for this mechanism. In a student essay, an unexplained `altChunk` is a high-value import clue. It is not misconduct by itself; it is evidence that content was brought in mechanically. **Detect:** search `word/document.xml`, `comments.xml`, `footnotes.xml`, `endnotes.xml`, headers, and footers for `w:altChunk`, then inspect the matching relationship in `.rels`. **Confidence:** high for import; medium for misconduct.

22

Red flag — hidden white text or similar obfuscation. Forensic work in academic integrity has shown documents containing white-coloured characters inserted to confuse plagiarism tools. In the XML, the exact representation can vary, but visible text may be split into unusual runs and formatting intended to conceal characters from normal reading while thwarting straightforward text matching. **Detect:** search for suspicious formatting patterns in runs, then render or recolour all text to expose hidden insertions. **Confidence:** high as an obfuscation clue. 23

Yellow flag — unusual run density, hyperlink residue, or heavy one-session cleanup after web paste. The literature reports cases where copied web text produced an unexpectedly large number of text runs and edit runs because hyperlinks and formatting had to be removed or normalised after paste. In a short document, extreme run density can therefore point to imported source material. **Detect:** inspect `word/document.xml` for `w:hyperlink` regions, count runs, and compare against the visible document length; inspect `document.xml.rels` for hyperlink relationships. **Confidence:** medium. 24

Yellow flag — abrupt language/locale shifts. OOXML allows run-level language settings with `w:lang`. Abrupt switches between locales—for example, body text apparently written in one national variety but runs marked with another, or multiple language clusters aligned with inserted sections—can indicate copied material, pasted citations, or cross-source assembly. This is still a weak clue because multilingual references and spell-check settings are common. **Detect:** inspect `styles.xml` and `document.xml` for `w:lang`, especially at run level. **Confidence:** low to medium. 25

Yellow flag — foreign or excessive style definitions. The styles part contains all style definitions and latent style information. Separate styles parts can even be replaced wholesale from another document. If a submission has a large custom style set, odd style IDs/names, or styles clearly tied to another organisation/template, that supports an import/template hypothesis. The same applies where the style inventory is disproportionately elaborate for a simple essay. **Detect:** inspect `word/styles.xml` and compare used styles against the body content. **Confidence:** medium. 26

Yellow flag — numbering definitions look imported or excessive. Numbering is stored in its own part and defines bullets, Arabic/Roman numbering, symbol bullets, and numbering instances. Imported fragments can drag in numbering definitions that have little to do with the visible text, or produce list structures inconsistent with the document's apparent simplicity. This is weaker than author/comment/RSID evidence, but it can help show document merging. **Detect:** inspect `word/numbering.xml`, count unused `abstractNum` / `num` structures, and compare list complexity against visible content. **Confidence:** low to medium. 27

Representative import/relationship pattern:

```
<!-- word/document.xml -->
<w:body xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/
```

```

main"
    xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/
relationships">
    <w:altChunk r:id="altChunk1"/>
    <w:p/>
</w:body>

<!-- word/_rels/document.xml.rels -->
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/
relationships">
    <Relationship Id="altChunk1"
        Type="http://schemas.openxmlformats.org/officeDocument/2006/
relationships/aFChunk"
        Target="import.html"/>
</Relationships>

```

This is a shortened representative pattern of the standard mechanism. 22

Footnotes, endnotes, headers, footers, images, relationships, embedded objects, and custom XML parts

Yellow to red flag — separate stories preserve material the body no longer shows.

WordprocessingML treats comments, headers, footers, footnotes, and endnotes as separate stories/parts. That matters because a user can clean the body while leaving institution names, version text, collaborator notes, prior headings, or copied source artefacts in those separate parts. Where those peripheral stories contradict the main body's apparent origin, the flag strengthens. **Detect:** inspect `header*.xml`, `footer*.xml`, `footnotes.xml`, and `endnotes.xml` rather than assuming the visible body is the whole document. **Confidence:** medium, rising when the peripheral text clearly points elsewhere. 28

Red flag — cropped images preserve hidden source context. Academic-forensics work has shown that the media extracted from a DOCX can retain the full original image even where the document displays only a cropped view. That can expose social-media UI, website chrome, usernames, timestamps, or other source-identifying material that the final visible crop tried to remove. **Detect:** inspect `/word/media/*` directly and compare stored images to rendered placement/cropping in the document. **Confidence:** high for image-source investigation. 29

Yellow flag — image alt text/title/description looks machine-generated, copied from the web, or tied to another source. Drawing objects can carry non-visual properties including `name`, `descr`, and `title`. Those fields can preserve source wording or accessibility text that does not match the document's style. This is particularly useful where the image was copied from elsewhere or inserted by an automated process. On its own it is a weak clue. **Detect:** inspect `wp:docPr` attributes in runs containing images or drawings. **Confidence:** low to medium. 30

Red flag — `.rels` points to suspicious targets. Relationships are the package's routing map. A relationship target can be inside or outside the package, and `MainDocumentPart` exposes external relationships and hyperlink relationships explicitly. In practice, `document.xml.rels` can reveal imported HTML/text, suspicious external hyperlinks, linked resources, embedded package parts, custom XML parts, and media targets even when the user-facing document obscures them. **Detect:**

inspect `/_rels/.rels`, `word/_rels/document.xml.rels`, and part-specific `.rels` files.

Confidence: high where the target directly identifies an imported source or external dependency. ³¹

Red flag — embedded objects or packages contain their own hidden provenance. The main document part explicitly supports embedded object parts and embedded package parts. A submission may therefore contain embedded spreadsheets, PDFs, or other packages carrying separate metadata, formulas, comments, or source material even though the surrounding essay appears simple. **Detect:** list package parts for `/embeddings/*`, inspect `w:object` /OLE references, and, where present, open the embedded package itself. **Confidence:** high if embedded content is relevant and unexplained. ³²

Yellow to red flag — unexpected custom XML/data binding suggests templated or automated generation. The main document part supports custom XML parts, and Office uses well-defined custom XML parts for document-management purposes. In a student essay, structured content controls or custom XML are not inherently suspicious, but unexplained data-bound placeholders, business metadata stores, or non-academic schemas can indicate a generated/template-driven origin. This is one of the more plausible DOCX-only clues for automation, though not specifically for a language model. **Detect:** inspect `/customXml/*`, relationship targets, and any `w:dataBinding` / content-control structures in document parts. **Confidence:** medium. ³³

Comparative table

The table below condenses the highest-value flags into a triage view. “Severity” means investigative urgency, not proof. “Confidence” means how useful the artefact is as a trigger for further investigation.

¹⁰

Field / part	Symptom	Detection method	Severity	Confidence	Remediation / next step
<code>docProps/core.xml</code>	Creator / last modifier does not match submitter or expected naming pattern	Read <code>dc:creator</code> , <code>cp:lastModifiedBy</code> , created/modified/revision	Yellow	Medium	Compare with student's other submissions and account naming conventions. ¹³
<code>docProps/core.xml</code> + assignment timeline	Created before brief release, or modified implausibly late with almost no apparent history	Correlate core dates with brief release/ submission time	Red	Medium-High	Escalate and compare with VLE/LMS timeline and draft history if available. ³⁴

Field / part	Symptom	Detection method	Severity	Confidence	Remediation / next step
<code>docProps/app.xml</code>	Very low <code>TotalTime</code> / low revision count for long document	Read <code>TotalTime</code> , <code>Words</code> , <code>Pages</code> , <code>Paragraphs</code> , <code>Application</code>	Yellow	Medium	Treat as supporting evidence only; look for RSID/import clues next. ³⁵
<code>docProps/app.xml</code>	Foreign template/company/app string inconsistent with claimed workflow	Inspect <code>Template</code> , <code>Company</code> , <code>Application</code> , <code>AppVersion</code>	Yellow	Medium	Compare against institutional templates and the student's normal environment. ³⁶
<code>docProps/custom.xml</code> / <code>customXml/*</code>	External organisational metadata, content-type IDs, business schemas	Inspect custom props and custom XML namespaces	Yellow-Red	Medium	Identify namespace origin; compare with known-good departmental files. ³⁷
<code>word/comments.xml</code> / revision markup	Third-party author, initials, dates, or reviewer names preserved	Search comments and <code>w:ins</code> / <code>w:del</code>	Red	High	Preserve screenshots/XML extracts and escalate immediately. ²¹
<code>word/settings.xml</code> / RSIDs	Shared <code>rsidRoot</code> with another suspicious file, or long bulk text with few RSID sessions	Inspect <code>w:rsidRoot</code> , RSID list, RSID-bearing attributes	Red	High for common-source comparison; Medium for single-file inference	Compare across related submissions and exemplars. ³⁸
<code>word/document.xml</code>	<code>w:altChunk</code> import anchor present	Search for <code>w:altChunk</code> , inspect matching relationship	Red	High for import	Determine what was imported and whether it fits the task. ²²

Field / part	Symptom	Detection method	Severity	Confidence	Remediation / next step
<code>word/document.xml</code>	Hidden white text / obfuscation formatting	Search suspicious runs and recolour/render all text	Red	High	Treat as a deliberate concealment clue; inspect source-matching next. ²³
<code>word/styles.xml</code> / <code>word/numbering.xml</code>	Excess custom styles or numbering definitions inconsistent with visible essay	Inspect style IDs/names and list definitions	Yellow	Low-Medium	Use as support for “imported/template-derived” hypothesis only. ³⁹
Headers / footers / notes	Peripheral stories contain leftover names, institutions, URLs, or comments	Inspect <code>header*.xml</code> , <code>footer*.xml</code> , <code>footnotes.xml</code> , <code>endnotes.xml</code>	Yellow-Red	Medium	Compare against institutional exemplars and body text. ⁴⁰
Media / drawings	Full uncropped image reveals hidden source context; alt text/title inconsistent with document voice	Inspect <code>/word/media/*</code> and <code>wp:docPr</code>	Red for cropped-source reveal; Yellow for alt text	High for image provenance; Low-Medium for alt text	Preserve original media and perform reverse-image/source comparison if justified. ⁴¹
<code>.rels</code> files	External targets, import parts, suspicious hyperlinks, unexpected custom XML or embedded-package relationships	Inspect package and part relationships	Red	High	Resolve each relationship target and document what it links to. ³¹

Field / part	Symptom	Detection method	Severity	Confidence	Remediation / next step
Embedded packages / objects	Hidden spreadsheet/PDF/object with separate provenance	List <code>/embeddings/*</code> , inspect <code>w:object</code> and package parts	Red	High	Extract and inspect the embedded package as a separate exhibit. ³²

Recommended tools and commands

For open-source triage, the most practical stack is a ZIP utility plus XML tooling. Use a ZIP-aware tool to list parts and timestamps; use ExifTool ⁴² for fast property extraction; use xmllint ⁴³ or XMLStarlet ⁴⁴ for XPath, formatting, and selective extraction; and use python-docx ⁴⁵ only when you need scripted inspection beyond shell triage. For schema/conformance checking, Open XML SDK ⁴⁶ is useful, but validator success says nothing about authorship or originality. ⁴⁷

A concise, repeatable command-line workflow looks like this:

```
# 1) List package contents
unzip -l suspect.docx

# 2) Inspect ZIP member timestamps / packaging clues
zipinfo -v suspect.docx

# 3) Dump high-value metadata parts
unzip -p suspect.docx docProps/core.xml | xmllint --format -
unzip -p suspect.docx docProps/app.xml | xmllint --format -
unzip -p suspect.docx docProps/custom.xml | xmllint --format -

# 4) Search for revision/import/author clues
unzip -p suspect.docx word/document.xml | grep -E "altChunk|w:ins|w:del|rsid|w:lang"
unzip -p suspect.docx word/comments.xml | grep -E "author|initials|date"

# 5) Inspect relationships
unzip -p suspect.docx word/_rels/document.xml.rels | xmllint --format -

# 6) Inspect peripheral stories and styles
unzip -p suspect.docx word/styles.xml | xmllint --format -
unzip -p suspect.docx word/numbering.xml | xmllint --format -
unzip -p suspect.docx word/footnotes.xml | xmllint --format -
unzip -p suspect.docx word/endnotes.xml | xmllint --format -
unzip -p suspect.docx word/header1.xml | xmllint --format -
unzip -p suspect.docx word/footer1.xml | xmllint --format -
```

If you want a scripted evidence schedule rather than raw XML, use Python's ZIP support to enumerate parts and write a small parser over the extracted XML. That approach is especially useful for counting

RSIDs, extracting all `w:lang` values, listing all style IDs actually used in the body, or producing a cross-document comparison of `rsidRoot`, comment authors, template/application strings, and relationship targets. ⁴⁸

Open questions and limitations

Several artefacts are volatile or workflow-dependent. Saving in a different editor, exporting, re-zipping, or copying content into a fresh file can rewrite package timestamps, strip RSIDs, normalise styles, or remove comments. Likewise, author/date information may be deliberately scrubbed by `removePersonalInformation` and `removeDateAndTime`. That means absence of evidence in DOCX structures is not evidence of absence. ⁴⁹

The most important limitation for AI investigations is conceptual. DOCX artefacts can often show that a file was assembled quickly, imported mechanically, templated, or edited by someone else. They rarely prove that prose came from a language model rather than from a human ghost-writer, copied source, transcription service, or ordinary template automation. Where AI use is the concern, DOCX forensics should be paired with source comparison, process evidence, oral verification, and, where policy permits, comparison against the student's own authored work. ⁸

The most defensible investigative practice is therefore comparative, not absolute: compare the suspect file against known-good submissions from the same course and template, against the student's prior work, and—if a common-source hypothesis exists—against other suspicious files for shared RSID roots, style inventories, comment authors, custom XML namespaces, embedded media, or relationship targets. That is where DOCX structure is most probative. ⁵⁰

-
- ¹ ⁹ ¹¹ ³¹ ⁴⁶ **Open Packaging Conventions Fundamentals | Microsoft Learn**
<https://learn.microsoft.com/en-us/previous-versions/windows/desktop/opc/open-packaging-conventions-overview>
- ² ⁷ ¹² ¹³ ³⁴ ⁴⁵ **IPackageProperties Interface (DocumentFormat.OpenXml.Packaging) | Microsoft Learn**
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.packaging.ipackageproperties?view=openxml-3.0.1>
- ³ ¹⁸ ²¹ ⁴⁴ **Comment Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn**
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.comment?view=openxml-3.0.1>
- ⁴ ⁸ ²³ ²⁴ ²⁹ ⁴¹ **Using digital forensics in higher education to detect academic misconduct | International Journal for Educational Integrity | Springer Nature Link**
<https://link.springer.com/article/10.1007/s40979-022-00104-1>
- ⁵ ³² **EmbeddedObject Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn**
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.embeddedobject?view=openxml-3.0.1>
- ⁶ ¹⁰ **Digital Investigation Techniques: A NIST Scientific Foundation Review**
<https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8354.pdf>
- ¹⁴ ¹⁵ ³⁵ ³⁶ ⁴³ **Properties Class (DocumentFormat.OpenXml.ExtendedProperties) | Microsoft Learn**
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.extendedproperties.properties?view=openxml-3.0.1>
- ¹⁶ ³⁷ **CustomDocumentProperty Class (DocumentFormat.OpenXml.CustomProperties) | Microsoft Learn**
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.customproperties.customdocumentproperty?view=openxml-3.0.1>

- 17 42 Document Information Panel and Document Properties in SharePoint Server 2010 (ECM) | Microsoft Learn
<https://learn.microsoft.com/en-us/previous-versions/office/developer/sharepoint-2010/bb447589%28v%3Doffice.14%29>
- 19 49 RemovePersonalInformation Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.removepersonalinformation?view=openxml-3.0.1>
- 20 38 50 RsidRoot Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.rsidroot?view=openxml-3.0.1>
- 22 AltChunk Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.altchunk?view=openxml-3.0.1>
- 25 Languages Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.languages?view=openxml-3.0.1>
- 26 39 Styles Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.styles?view=openxml-3.0.1>
- 27 Numbering Class (DocumentFormat.OpenXml.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.wordprocessing.numbering?view=openxml-3.0.1>
- 28 40 Structure of a WordprocessingML document | Microsoft Learn
<https://learn.microsoft.com/en-us/office/open-xml/word/structure-of-a-wordprocessingml-document>
- 30 DocProperties Class (DocumentFormat.OpenXml.Drawing.Wordprocessing) | Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.drawing.wordprocessing.docproperties?view=openxml-3.0.1>
- 33 [MS-OI29500]: Office Well Defined Custom XML Parts | Microsoft Learn
https://learn.microsoft.com/en-us/openspecs/office_standards/ms-oi29500/b5ea0047-2119-4ae5-94a1-63c27af037dd
- 47 exiftool Application Documentation
https://exiftool.org/exiftool_pod.html?utm_source=chatgpt.com
- 48 zipfile — Work with ZIP archives
https://docs.python.org/3/library/zipfile.html?utm_source=chatgpt.com